TP n 5

Bases de données Travaux Pratiques

AZZOUNI Mohamed Aziz

27 mars 2026

Ex1

Commencer par ouvrir deux sessions (S1 et S2) et exécuter la commande suivante dans chacune des sessions :

```
SET AUTOCOMMIT OFF
```

Question 1. Créer la table `transaction` en utilisant la session S1.

```
CREATE TABLE transaction (
    idTransaction VARCHAR 2(44),
    valTransaction NUMBER (10)
);
```

Question 2. Session S2 : insertions, modification, suppression puis `ROLLBACK`.

```
-- Insertions
INSERT INTO transaction VALUES ('T001', 100);
INSERT INTO transaction VALUES ('T002', 200);
INSERT INTO transaction VALUES ('T003', 300);

-- Modification
UPDATE transaction SET valTransaction = 999 WHERE idTransaction = 'T001'
    ;

-- Suppression
DELETE FROM transaction WHERE idTransaction = 'T003';

-- Annulation de toutes les modifications
ROLLBACK ;

-- Verification
SELECT * FROM transaction ;
```

Observation attendue : Après `ROLLBACK`, la table est vide : toutes les modifications sont annulées. Le `ROLLBACK` restaure l' tat initial de la base.

Question 3. Session S2 : insertions puis fermeture avec `quit` sans `COMMIT`.

```
-- Session S2 : nouvelles insertions
INSERT INTO transaction VALUES ('T010', 500);
INSERT INTO transaction VALUES ('T011', 600);
```

```

-- Fermeture sans COMMIT explicite
quit ;
```

Observation attendue : En quittant avec `quit;` sans avoir mis de `COMMIT`, Oracle effectue un **ROLLBACK automatique**. La session S1 ne voit donc pas les données insérées par S2 : elles ont t perdues.

Question 4. Session S1 : insertions puis fermeture brutale de sqlplus.

```
-- Session S1 : insertions avant fermeture brutale
INSERT INTO transaction VALUES ('T020', 700);
INSERT INTO transaction VALUES ('T021', 800);

-- Fermeture brutale (CTRL+C ou fermeture fenetre)
-- Reconnexion et verification
SELECT * FROM transaction;
```

Observation attendue : Les données ne sont pas préservées. Une fermeture brutale déclenche un **ROLLBACK implicite** : toute transaction non validée est annulée.

Question 5. Insertion + `ALTER TABLE` + `ROLLBACK`.

```
-- Insertions
INSERT INTO transaction VALUES ('T030', 900);

-- Modification de structure (DDL)
ALTER TABLE transaction ADD val2transaction NUMBER(10);

-- Tentative de ROLLBACK
ROLLBACK;

-- Verification
SELECT * FROM transaction;
DESCRIBE transaction;
```

Observation attendue : L'instruction `ALTER TABLE` est un ordre DDL. Dans Oracle, tout DDL provoque un **COMMIT implicite** avant et après son exécution. Le `ROLLBACK` ne peut donc plus annuler ni les insertions précédentes, ni l'altération de la structure.

Question 6. Conclusion D nitions.

Session : Une session est une connexion active entre un utilisateur et le serveur de base de données. Elle d bute la connexion et se termine la d connexion. Une session peut contenir plusieurs transactions exécutées en s rie.

Transaction : Une transaction est une séquence d'op rations (lectures, é critures) trait e comme une unit atomique. Elle commence implicitement après la dernière validation (ou au d but de la session) et se termine par un `COMMIT` ou un `ROLLBACK`.

Valider une transaction : `COMMIT` rend les modifications permanentes et visibles par les autres sessions.

Annuler une transaction : ROLLBACK annule toutes les modi cations e ectu es depuis le dernier COMMIT, restaurant l' tat pr c dent de la base.

Propri t s ACID :

Atomicit : une transaction s'ex cute en totalit ou pas du tout (tout ou rien).

Coh rence : la base reste dans un tat coh rent avant et apr s chaque transaction.

Isolation : les transactions s'ex cutent de fa on isol e ; les modi cations non valid es sont invisibles pour les autres sessions.

Durabilit : apr s un COMMIT, les donn es sont d nitivement enregistr es, m me en cas de panne.

Ex2

2.1 Création du schéma

```
CREATE TABLE vol (
    idVol                    VARCHAR2 (44) ,
    capacite Vol             NUMBER (10) ,
    nbr Places Reservees Vol NUMBER (10)
);

CREATE TABLE client (
    id Client                VARCHAR2 (44) ,
    prenom Client            VARCHAR2 (11) ,
    nbr Places Reservees Cleint NUMBER (10)
);

INSERT INTO vol VALUES ('V001', 10, 0);
INSERT INTO client VALUES ('C001', 'Alice', 0);
INSERT INTO client VALUES ('C002', 'Bob', 0);
COMMIT ;
```

2.2 Isolation des transactions

```
-- Session S1 (T1) : reservation de 2 billets pour Alice
UPDATE vol SET nbr Places Reservees Vol = nbr Places Reservees Vol + 2
    WHERE idVol = 'V001';
 UPDATE client SET nbr Places Reservees Cleint = nbr Places Reservees Cleint +
    2 WHERE idClient = 'C001';

-- Ne pas valider encore (pas de COMMIT )

-- Session S2 (T2) : lecture de l'etat de la base
SELECT * FROM vol ;
SELECT * FROM client ;
-- => T2 voit toujours l'etat AVANT les modifications de T1
```

Les mises jour de T1 sont visibles par T1 elle-m me (coh rence interne) mais invisibles par T2. C'est le principe d'isolation des transactions : chaque transaction travaille sur sa propre vue coh rente de la base.

2.3 COMMIT et ROLLBACK

```sql
-- Session S1 (T1) : annulation
ROLLBACK ;

-- Verification dans S2 (T2)
SELECT * FROM vol ;
SELECT * FROM client ;
-- => La base est revenue a son etat initial
```

```sql
-- Session S1 (T1) : nouvelle reservation validee
UPDATE vol SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 2 WHERE idVol = 'V001';
UPDATE client SET nbrPlacesReserveesCleint = nbrPlacesReserveesCleint + 2 WHERE idClient = 'C001';
COMMIT ;

-- Session S2 (T2) : maintenant T2 voit les modifications
SELECT * FROM vol ;
SELECT * FROM client ;
```

Aprés COMMIT de T1 : un ROLLBACK n'a plus aucun e et (durabilité). T2 voit d sormais les mises jour de T1. Cela conrme que le niveau d'isolation par d faut d'Oracle est READ COMMITTED : chaque SELECT voit la dernière version validée au moment de son exécution.

2.4 Isolation incomplète mises jour perdues (Lost Update)

```sql
-- === Etape 1 : Selections T1 ===
-- S1
SELECT nbrPlacesReserveesVol FROM vol WHERE idVol = 'V001'; -- => 0
SELECT nbrPlacesReserveesCleint FROM client WHERE idClient = 'C001'; -- => 0

-- === Etape 2 : Selections T2 ===
-- S2
SELECT nbrPlacesReserveesVol FROM vol WHERE idVol = 'V001'; -- => 0
SELECT nbrPlacesReserveesCleint FROM client WHERE idClient = 'C002'; -- => 0

-- === Etape 3 : Mises a jour T1 + COMMIT ===
-- S1
UPDATE vol SET nbrPlacesReserveesVol = 0 + 2 WHERE idVol = 'V001';
UPDATE client SET nbrPlacesReserveesCleint = 0 + 2 WHERE idClient = 'C001';
COMMIT ;

-- === Etape 4 : Mises a jour T2 + COMMIT ===
-- S2 (utilise les valeurs lues a l'etape 2, donc 0 + 3 = 3, ignorant T1)
UPDATE vol SET nbrPlacesReserveesVol = 0 + 3 WHERE idVol = 'V001';
UPDATE client SET nbrPlacesReserveesCleint = 0 + 3 WHERE idClient = 'C002';
```

```
COMMIT ;
```

Observation attendue : Les deux clients ont r serv 2+3 = 5 billets, mais nbrPlacesReserveesVol ne vaut que 3 (et non 5). La mise jour de T1 a t cras e par T2 : c'est le probl me des mises jour perdues (lost updates). La base est dans un tat incoh rent.

2.5 Isolation complète mode SERIALIZABLE

```
-- Debut de session avec isolation SERIALIZABLE
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE ;
```

Observation attendue : L'une des deux transactions sera rejet e avec l'erreur ORA-08177 : can't serialize access for this transaction. La coh rence est garantie, au prix d'un rejet ventuel qu'il faudra g rer applicativement (relancer la transaction).

2.6 Exécution concurrente vue en cours : $r_1(d)$ $w_2(d)$ $w_2(d')$ C_2 $w_1(d')$ C_1

```
-- S1 (T1) : lecture de d
SELECT nbrPlacesReserveesVol FROM vol WHERE idVol = 'V001';

-- S2 (T2) : ecriture sur d et d', puis COMMIT
UPDATE vol SET nbrPlacesReserveesVol = 5 WHERE idVol = 'V001'; -- w2(d)
UPDATE vol SET capaciteVol = 8           WHERE idVol = 'V001'; -- w2(d')
COMMIT ;                                                       -- C2

-- S1 (T1) : ecriture sur d', puis COMMIT
UPDATE vol SET capaciteVol = 10 WHERE idVol = 'V001'; -- w1(d')
COMMIT ;                                              -- C1
```

Conclusion L'algorithme Oracle est-il le m me que le 2PL vu en cours ?

Non. Oracle n'impl mente pas le verrouillage deux phases (2PL) strict tel qu'enseign en cours. Il utilise le MVCC (Multi-Version Concurrency Control) :

En 2PL strict : chaque transaction pose un verrou (partag ou exclusif) sur chaque donn e lue ou crite, et ne lib re aucun verrou avant COMMIT/ROLLBACK. Cela garantit la s rialisabilit mais peut provoquer des interblocages (deadlocks).

En MVCC (Oracle) : les lecteurs voient une version coh rente de la ligne sans bloquer les crivains. Seules les critures posent des verrous exclusifs.

En READ COMMITTED (d faut) : chaque SELECT voit la derni re version valid e au moment de son ex cution ⇒ risque de lost update.

En SERIALIZABLE : chaque SELECT voit la version valid e au d but de la transaction ; Oracle l ve ORA-08177 en cas de con it.

La s quence $r_1(d)$ $w_2(d)$ $w_2(d')$ C_2 $w_1(d')$ C_1 est accept e par Oracle en READ COMMITTED : T1 n'a pas pos de verrou sur d lors de sa lecture, T2 peut donc crire librement. En 2PL strict, T1 aurait bloqu T2 d s $r_1(d)$ (verrou partag), ce qui aurait chang le r sultat. Oracle privil gie ainsi la performance sur la stricte s rialisabilit en mode READ COMMITTED.